

When Does Prompt Elaboration Stop Helping? A Controlled Study of Prompt Sensitivity Across LLM Tasks

Anonymous Author(s)

ANONYMOUS@EXAMPLE.COM

Anonymous Institution

Editors: TBD

Abstract

Practitioners routinely craft increasingly verbose prompts under the assumption that more context yields better responses. We test this assumption with a controlled study spanning 17,378 LLM-evaluated responses across seven large language models, six tasks, and seven additive prompt-elaboration levels. Our methodology isolates the effect of prompt content from prompt length through three complementary experiments: (1) a core saturation study in which each prompt level adds exactly one elaboration layer (task label, format specification, definitions, persona, guidelines, worked example); (2) a randomised layer-ordering ablation (2,986 runs) that shuffles the introduction order of prompt layers; and (3) an irrelevant-padding control (1,470 runs) that matches token counts with meaningless filler. We find that response quality is governed by *information sufficiency*, not token count: quality saturates once task-critical content has been communicated, regardless of when it appears in the prompt or how many tokens surround it. Tasks vary in a four-level *prompt-sensitivity gradient*: *concentrated* (classification—49% of quality gain from one layer), *distributed* (product extraction—gains across multiple layers), *diffuse* (instruction following—small gains across many layers), and *insensitive* (QA, math, summarisation—at ceiling from the bare input). An independent second judge (gemini-2.0-flash, 3,915 re-evaluations) confirms the primary patterns. Model scale modulates the effect: smaller models (1B–8B) require more prompt elaboration and benefit more from it, but saturate at lower quality. These findings provide concrete, task-specific guidance for prompt authoring: for classification, specify the task label; for structured extraction, include a worked example; for QA and math, the bare question suffices.

Keywords: prompt engineering; LLM evaluation; prompt saturation; information sufficiency; prompt sensitivity

1. Introduction

The rapid adoption of large language models (LLMs) in production systems has generated a body of prompt engineering heuristics: add role descriptions, include worked examples, append step-by-step instructions, and specify output formats in detail (Brown et al., 2020; Wei et al., 2022; Zhou et al., 2023). Official guidance from major providers recommends structured prompts with role, objective, instructions, reasoning steps, output format, examples, and context—each section adding tokens (Sahoo et al., 2024; Schulhoff et al., 2024). These practices implicitly assume a monotone relationship between prompt elaboration and response quality. But is this assumption empirically justified, and if so, *up to what point*?

Token costs are not merely economic. Longer prompts consume more inference compute, fill context windows faster, and can *degrade* performance when key information is buried in irrelevant context (Liu et al., 2024). The prompt compression literature (Jiang et al., 2023;

Pan et al., 2024) has demonstrated that many tokens are expendable without quality loss, but these methods operate on already-verbose prompts and assume compression is safe. A more fundamental question is: *when does prompt elaboration stop helping in the first place?*

We address this through a controlled **prompt sensitivity study**. For each of six tasks we construct seven strictly additive prompt templates: each template is a proper superset of the previous, adding exactly one *mechanism* of elaboration (task label, output format, field definitions, persona, guidelines, worked example). We run every (model, task, level, example) combination for seven frontier LLMs and fit quality-versus-token curves to measure where quality plateaus. Critically, we disentangle information content from token count through two complementary controls: a randomised layer-ordering ablation that shuffles which mechanism appears at each token count, and an irrelevant-padding experiment that matches token counts with meaningless filler.

Our results show that quality is governed by *information sufficiency*—the curve bends once task-critical content has been communicated, irrespective of position or raw token count. Tasks differ not in whether they saturate, but in *how* their quality gain is distributed across prompt mechanisms. We describe this as a **prompt-sensitivity gradient**:

- **Concentrated:** classification—49% of quality gain from one layer (task label).
- **Distributed:** product extraction—gains accumulate across multiple layers (task label 32%, worked example 40%).
- **Diffuse:** instruction following—small gains spread across many layers.
- **Insensitive:** QA, math reasoning, summarisation—near-zero or negative total gain; models are already at ceiling from the bare input.

Contributions.

1. A reusable **additive prompt-level methodology** with randomised ablation and padding controls for measuring prompt sensitivity, with open-source code and data.
2. Empirical evidence that prompt saturation is driven by **information sufficiency**, not token count, demonstrated through ablation (2,986 runs) and padding control (1,470 runs) experiments.
3. A **prompt-sensitivity gradient** characterising how quality gain distributes across prompt mechanisms, from concentrated (one critical layer) to insensitive (already at ceiling).
4. **Dual-judge validation:** an independent second judge (3,915 re-evaluations) confirms the primary patterns where signal exists and reveals ceiling compression as the source of apparent disagreement.
5. **Capability-modulated saturation** across model scales from 1B to frontier API models: smaller models are more prompt-sensitive—they require more elaboration and benefit more from it—but saturate at lower asymptotic quality.

The total experimental scope is 17,378 LLM evaluations across the main study, replication, ablation, padding control, second-judge validation, and sub-8B experiments. All prompts, evaluation code, and data are included in the supplementary material.

2. Related Work

Prompt compression. LLMingua (Jiang et al., 2023) and LLMingua-2 (Pan et al., 2024) show that prompts can be compressed 3–20× with minimal quality loss, demonstrating substantial token redundancy. Chevalier et al. (2023) train models to summarise prompts into soft tokens. More recent pipelines such as CompactPrompt (Choi et al., 2025) combine token pruning with n-gram abbreviation for end-to-end compression. These methods operate *at inference time, per instance*—compressing a written prompt by removing low-information tokens. Our work operates *at authoring time, per task*: we provide priors on which prompt mechanisms yield quality gains for a given task type, before any specific instance is constructed. The two approaches are complementary: author the right mechanisms (our work), then compress the wording within them (LLMingua).

Prompt engineering. Chain-of-thought (Wei et al., 2022) and few-shot (Brown et al., 2020) prompting add tokens in exchange for quality gains on reasoning tasks. Zhou et al. (2023) show that task-description quality matters more than length for many tasks. Recent surveys of prompting strategies (Sahoo et al., 2024; Schulhoff et al., 2024) and automatic prompt optimisation (Li et al., 2025) catalogue dozens of techniques but do not measure their marginal effect as a function of token count. Our work complements these by quantifying the return of each successive elaboration layer and showing that marginal returns vary systematically across task types.

Long-context performance. Liu et al. (2024) demonstrate that LLMs underutilise information positioned in the middle of long contexts, a degradation effect distinct from our finding, which concerns the marginal benefit of additional *instruction* tokens before any degradation. Levy et al. (2024) and Hsieh et al. (2024) study how performance varies with context length on retrieval and reasoning benchmarks, finding similar non-monotone effects at much larger scales.

Prompt sensitivity and robustness. Webson and Pavlick (2022) and Mizrahi et al. (2024) show that seemingly small prompt changes cause large quality swings. Our design controls for this by holding content constant and varying only elaboration depth, isolating the information-content signal from phrasing effects. The randomised ablation further controls for ordering effects.

Scaling laws. Neural scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) describe how model quality grows with compute and training data. We study an analogous phenomenon at the *prompt level*: how quality grows with instruction tokens, and where this growth saturates. To our knowledge, no prior work has fitted and statistically tested saturation curves across multiple tasks, models, and prompt orderings in a controlled additive design.

LLM-as-judge evaluation. Zheng et al. (2023) show that strong LLM judges match human preferences at over 80% agreement, establishing the viability of automated evaluation. Recent work has shown that different LLM judges exhibit systematically different score calibrations (Chen et al., 2025; Li et al., 2026), motivating our dual-judge design with both gpt-4o-mini and gemini-2.0-flash.

Table 1: Additive prompt layers by task. Each row adds content to all previous rows. Layers represent qualitatively different prompt mechanisms, not homogeneous units of length.

Lv	Mechanism	Classification	Prod. Extraction	QA	Math Reasoning	Summarisation	Instr. Following
1	Bare input	Raw text only	Product text	Question only	Problem statement	Article text	Topic prompt
2	Task label	“Classify sentiment”	List field names	“Answer the question”	“Solve the problem”	“Summarise this”	Task + constraints
3	Format spec	“Pos/Neg/Neu”	JSON format	“Answer concisely”	“Give a number”	Length target	Format specification
4	Definitions	Label definitions	Field definitions	Domain context	Show-work instr.	Content guidance	Constraint definitions
5	Persona	Expert analyst	Expert extractor	Domain expert	Math tutor	Journalist	Writing expert
6	Guidelines	Mixed/ironic rules	Edge cases	Uncertainty	Step-by-step rules	Quality rules	Formatting rules
7	Example	Worked example	Worked example	Worked example	Worked example	Worked example	Worked example

3. Methodology

3.1. Additive Prompt Levels

For each task we define seven prompt templates $T_1 \subset T_2 \subset \dots \subset T_7$, where $\subset$ denotes strict token superset. Each level appends one layer of elaboration—a distinct prompt *mechanism*—to all previous content; no layer modifies earlier text. Table 1 shows the layer structure for all six tasks.

The additive design is critical: because level $k + 1$ contains all tokens of level k plus exactly one new mechanism, any quality difference between adjacent levels is attributable solely to that mechanism. No layer is filler; every added layer is meaningful instruction that could plausibly improve performance.

3.2. Tasks

We study six tasks spanning a spectrum of output structure:

- **Classification:** sentiment classification (positive / negative / neutral) over 20 product reviews and social media posts. Heuristic evaluator: correct label present in response.
- **Product extraction:** extract name, price, brand, category from product listings in JSON format. 20 examples with varying difficulty (buried prices, implicit brand names). Heuristic: fraction of four fields correctly matched.
- **Question answering (QA):** factoid questions across geography, science, and history. 20 examples with ground-truth strings. Heuristic: word-overlap recall (ROUGE-1).
- **Math reasoning:** 2–3 step arithmetic, rate, and geometry problems. 20 examples (10 easy, 10 medium). Heuristic: exact numerical match on the last number in the response.
- **Summarisation:** abstractive summarisation of news passages. 20 examples with reference summaries. Heuristic: length score + ROUGE-1 recall.
- **Instruction following:** responses must satisfy explicit structural constraints (bullet points, numbered list, single word). 20 examples, each with 1–2 constraints. Heuristic: fraction of constraints satisfied.

Table 2: Models evaluated. Parameter counts are listed where publicly disclosed; “undiscl.” indicates the provider has not released this information. †Evaluated on classification and product extraction only (see text).

Model (alias)	Provider	API	Parameters
llama-3.2-1b-instruct†	Meta	Groq	1B
llama-3.1-8b-instant	Meta	Groq	8B
qwen/qwen3-32b	Alibaba	Groq	32B
llama-3.3-70b-versatile	Meta	Groq	70B
gemini-2.0-flash	Google	Gemini	undiscl.
moonshotai/kimi-k2	Moonshot	Groq	undiscl.
gpt-4o-mini	OpenAI	OpenAI	undiscl.
claude-haiku-4-5	Anthropic	Anthropic	undiscl.

To assess robustness to dataset size, we replicate the experiment on 200 examples for classification (SST-2; Socher et al. 2013) and QA (SQuAD; Rajpurkar et al. 2016), preserving the same prompt templates and evaluation procedure. These two tasks represent opposite poles of the prompt-sensitivity gradient (strongest saturation vs. no saturation); confirming both the positive and negative findings at larger scale bounds the intermediate tasks. Product extraction receives independent confirmation through the ablation (2,986 runs with new orderings and examples) and padding control (1,470 runs) rather than a separate replication.

3.3. Models

We evaluate eight LLMs across four providers, spanning 1B to 70B disclosed parameters plus four API-tier models with undisclosed sizes (Table 2). All models are accessed via production APIs with default temperature settings. Each response is capped at 512 output tokens. The seven primary models run all six tasks; Llama-3.2-1B is evaluated on classification and product extraction only—the two tasks where saturation is present and the parameter-scale interaction is testable—to extend the size range below 8B without inflating the experiment matrix.

3.4. Evaluation

Primary judge. Following Zheng et al. (2023), all responses are scored by gpt-4o-mini on four dimensions (correctness, completeness, reasoning, conciseness), each rated 1–5. The aggregate quality score is the normalised sum: $q = \frac{\sum_i s_i}{20} \in [0, 1]$. We provide task-specific rubrics to the judge (e.g., for classification: focus on whether the label matches; for product extraction: whether all four fields are present and correct).

Independent second judge. To assess judge reliability, all 3,915 main-study responses for the four tasks with the strongest and weakest saturation signals (classification, math reasoning, QA, product extraction) are re-evaluated by gemini-2.0-flash using the identical rubric.

Heuristic validation. We validate the LLM judges against task-specific heuristic evaluators on the full dataset (Pearson $r = 0.986$ for the primary judge), confirming that judge scores are reliable.

Aggregation. For each (model, task, level) triplet, we aggregate quality across all 20 examples by taking the mean, yielding 7 data points per (model, task) curve.

3.5. Curve Fitting and Saturation Points

For each (model, task) pair we fit two functional forms to the 7 level means:

$$q_{\log}(x) = a \log(bx) + c, \quad q_{\text{sig}}(x) = \frac{L}{1 + e^{-k(x-x_0)}} + c \quad (1)$$

where x is the mean prompt token count at each level. We select the form with lower RMSE. The **saturation point** T^* is defined as the smallest x such that:

$$q(x) \geq 0.95 \times \max_x q(x) \quad (2)$$

Null model F-test. To test whether a fitted curve is significantly better than a flat line ($q(x) = \bar{q}$), we compute:

$$F = \frac{(SS_{\text{null}} - SS_{\text{fit}})/(k - 1)}{SS_{\text{fit}}/(n - k)} \quad (3)$$

where $k \in \{3, 4\}$ is the number of curve parameters, $n = 7$ is the number of level means, and $\alpha = 0.05$.

Bootstrap confidence intervals. We estimate 95% CIs on T^* via 1,000-iteration bootstrap. On each iteration we resample the 20 example indices *with replacement* and apply the same resample across all seven levels, preserving paired structure.

Threshold sensitivity. To assess robustness of saturation estimates, we recompute T^* at four threshold levels (85%, 90%, 95%, 99% of the fitted asymptote) and using a threshold-free second-derivative knee estimator.

3.6. Marginal Contribution Analysis

For each task, we compute the quality delta between adjacent level pairs ($\Delta_{k \rightarrow k+1} = \bar{q}_{k+1} - \bar{q}_k$), averaged across all models. This decomposes the total quality gain into per-mechanism contributions, revealing which prompt layers drive improvement and which are redundant. The marginal contribution profile characterises a task’s position on the prompt-sensitivity gradient.

3.7. Randomised Layer-Ordering Ablation

To disentangle information content from token count, we conduct a randomised ablation. For classification and product extraction—the tasks with significant saturation in the core study—we shuffle the introduction order of all six prompt layers (layers 2–7; layer 1 is always the bare input), generating 5 random permutations per task. Each permutation is

Table 3: Experimental scale across all study components.

Component	Evaluations
Main study (7 models $\times$ 6 tasks $\times$ 7 levels $\times$ 20 examples)	5,875
200-example replication (classification + QA)	$\approx$ 2,800
Randomised layer-ordering ablation	2,986
Irrelevant-padding control	1,470
Independent second judge (gemini-2.0-flash)	3,915
Sub-8B experiment (Llama-3.2-1B)	332
Total	17,378

run on 7 curated examples per task across 6 models (2,986 total runs; 2,450 analysed after excluding kimi-k2 due to API deprecation between study phases).

If saturation is driven by token count, shuffling should not affect the quality-versus-token curve. If saturation is driven by information content, the saturation *point* (in tokens) should shift with ordering—because high-value layers can now appear early—while the saturation *shape* (quality rising then plateauing) should persist.

3.8. Irrelevant-Padding Control

As a direct test of the token-count null hypothesis, we replace real prompt content with irrelevant filler matched to the same token counts. Three padding types are used: factual trivia unrelated to the task (geography, history, science), repeated neutral phrases, and random English words in shuffled order. The bare task instruction is preserved as the only meaningful content. This yields 1,470 runs (5 models $\times$ 2 tasks $\times$ 3 padding types $\times$ 7 levels $\times$ 7 examples).

If tokens themselves drive quality, padded prompts should show the same saturation curve as real prompts at matched token counts.

3.9. Experimental Scale

Table 3 summarises the total experimental scope.

4. Results

4.1. Quality-versus-Token Curves Reveal Task-Dependent Saturation

Figure 1 shows quality-versus-token curves for all six tasks and seven models. Classification curves rise steeply at low token counts and flatten well before level 7, with four of seven models producing F-test-significant saturating curves ($p < 0.05$; Table 4). Saturation points for these four models range from 42 to 64 tokens, well below a typical worked-example prompt (level 7: $\approx$ 300–400 tokens).

Product extraction curves are similarly sigmoidal: three of seven models are significant, with saturation points ranging from 92 (llama-3.3-70b) to 536 (claude-haiku) tokens. The wider range reflects greater schema complexity: a four-field JSON extraction requires more

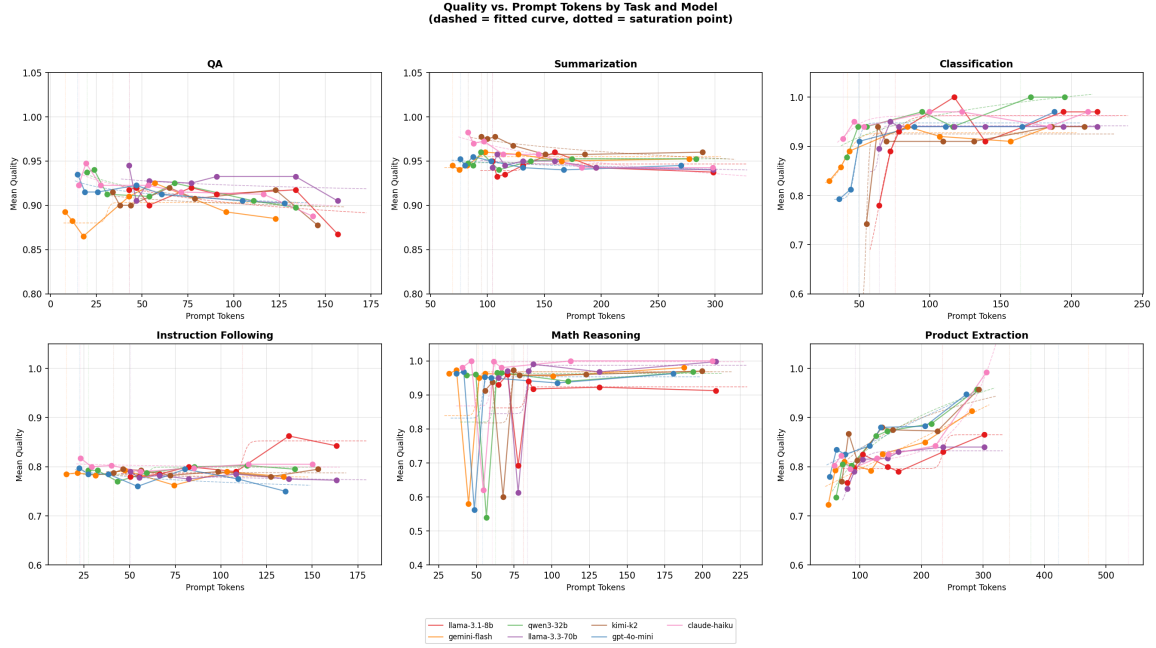


Figure 1: Quality (LLM judge, normalised to $[0,1]$) vs. mean prompt tokens across seven levels, for all six tasks and seven models. Dashed lines show the best-fit log/sigmoid curve; dotted verticals mark the 95%-of-asymptote saturation point. Classification and product extraction show clear sigmoid curves; QA and math reasoning are flat.

elaboration cues than binary sentiment labelling. For all significant product extraction pairs, $R^2 > 0.70$, confirming strong curve fit.

QA and math reasoning yield zero significant pairs out of seven models each. Fitted curves have low R^2 (QA: 0.06–0.69; math: 0.12–0.34) and fail to beat the flat null model at $\alpha = 0.05$ in every case. This is a *meaningful negative result*: the same experimental setup detects strong saturation in classification, ruling out methodological artefact.

Summarisation (1/7 significant, claude-haiku only) and instruction following (1/7 significant, llama-3.1-8b only) occupy an intermediate position.

4.2. Ablation Confirms Information Content, Not Token Count

The randomised layer-ordering ablation tests whether the saturation curve is driven by information content or raw token count. Table 5 summarises the product extraction results.

The three models that are most prompt-sensitive in the main study—qwen3-32b, llama-3.1-8b, and llama-3.3-70b—show robust saturation across orderings: 13/15 fits are significant, with saturation points clustering at 65–140 tokens regardless of which layers come first. The more prompt-insensitive models (gemini-flash, claude-haiku) show flatter curves with fewer significant fits (1/10), consistent with these models reaching high quality from minimal prompting.

The ablation saturation points (65–140 tokens) are substantially lower than the original fixed-ordering estimates for the same models (92–378 tokens). This is expected: in the fixed

Table 4: F-test results and saturation points for the main study. Bold rows are significant ($p < 0.05$). CI: 95% bootstrap confidence interval on saturation token count T^* .

Task	Model	F	p	T^*	95% CI
Classification	gemini-flash	14.5	0.027	42	[29, 147]
	kimi-k2	21.1	0.016	57	[55, 237]
	gpt-4o-mini	42.0	0.006	50	[46, 230]
	llama-3.3-70b	22.3	0.015	64	[63, 72]
Product Extraction	llama-3.3-70b	16.4	0.023	92	[83, 312]
	qwen3-32b	26.6	0.005	378	[118, 506]
	claude-haiku	49.6	0.005	536	[233, 586]
Summarisation	claude-haiku	9.4	0.031	83	[78, 87]
Instr. Following	llama-3.1-8b	13.1	0.031	112	[50, 314]
QA (all 7)	All $p > 0.12$; not significant				—
Math (all 7)	All $p > 0.70$; not significant				—

 Table 5: Randomised ablation results for product extraction. Mean $\pm$ standard deviation of saturation points across 5 random permutations. Significant fits: number of permutations producing $p < 0.05$.

Model	T^* Mean $\pm$ Std (tokens)	Range	Sig. fits
qwen3-32b	95 $\pm$ 26	[65, 134]	5/5
llama-3.1-8b	110 $\pm$ 24	[84, 140]	4/5
llama-3.3-70b	110 $\pm$ 22	[80, 131]	4/5
gemini-flash	57 $\pm$ 17	[49, 91]	1/5
claude-haiku	58 $\pm$ 0	[58, 58]	0/5

ordering, the most valuable layer for product extraction—the worked example (contributing 40% of quality gain; see Section 4.4)—always comes last, so quality stays low until high token counts. In random orderings, this high-value layer can appear at any position, causing quality to rise sooner and saturation to occur at fewer tokens. The shift in saturation point directly demonstrates that it is *information content and ordering*, not raw token count, that determines where the curve bends.

Classification shows only 2/25 significant fits in the ablation—but this is expected given classification’s prompt-sensitivity profile. Classification’s quality gain is concentrated in a single layer (Section 4.4): once the task label appears at any position, quality jumps and the curve is flat thereafter. Curve fitting is the wrong tool for detecting a step function; the 2/25 result reflects a method-shape mismatch, not an absence of prompt sensitivity.

4.3. Padding Control Confirms the Token-Count Null

The irrelevant-padding experiment provides a direct test: do tokens themselves buy quality? Across 1,470 runs, quality with irrelevant padding remained flat or declined (mean L1→L7 $\Delta = -0.074$ for classification, -0.013 for product extraction). Only 1 of 26 conditions showed a significant positive trend (Spearman $p < 0.05$)—a near-ceiling case (claude-haiku

Table 6: Real prompt content vs. irrelevant padding: L1→L7 quality delta (Δ) per model and task. Positive values indicate quality improvement from L1 to L7.

Model	Task	Real Δ	Padding Δ
gemini-flash	classification	+0.110	+0.031
llama-3.3-70b	classification	+0.045	−0.221
llama-3.1-8b	classification	+0.190	−0.161
qwen3-32b	classification	+0.123	−0.029
claude-haiku	classification	+0.055	+0.029
gemini-flash	product extraction	+0.191	−0.002
llama-3.3-70b	product extraction	+0.085	−0.055
llama-3.1-8b	product extraction	+0.098	+0.043
qwen3-32b	product extraction	+0.220	−0.038
claude-haiku	product extraction	+0.190	−0.011

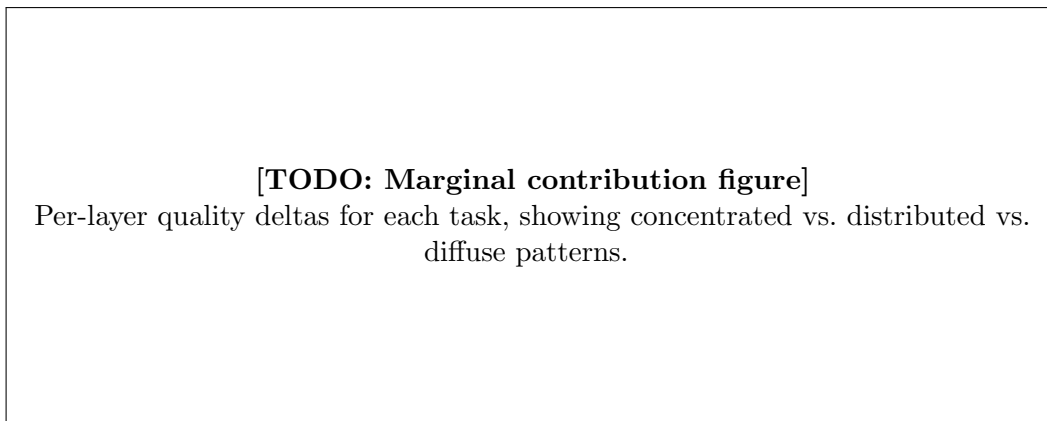


Figure 2: Per-layer marginal quality contributions ($\Delta_{k \rightarrow k+1}$) averaged across seven models. Classification gain is concentrated in L1→L2 (task label); product extraction gain is distributed across L1→L2 and L6→L7 (worked example); math reasoning shows a sharp L3 collapse from suppressing chain-of-thought.

classification, $\Delta = +0.03$, from 0.97 to 1.00) consistent with chance at 26 tests. Three of 26 significant trends were negative: padding slightly *hurt* quality.

In contrast, real prompt content over the same token range produced clear gains: +0.10 for classification and +0.16 for product extraction. Table 6 shows the per-model comparison. Real content exceeds padding for 9 of 10 model–task pairs, with the largest gaps in llama-3.1-8b classification (+0.190 vs. −0.161) and qwen3-32b product extraction (+0.220 vs. −0.038).

4.4. Marginal Contributions Reveal the Prompt-Sensitivity Gradient

Marginal contribution analysis decomposes quality gain into per-mechanism contributions, revealing how different tasks distribute their sensitivity across prompt layers. Figure 2 shows the per-layer quality deltas averaged across all seven models.

Classification (concentrated). L1→L2 (adding the task label) contributes +0.073 quality, accounting for 49% of total positive gain. Subsequent layers add diminishing returns: L2→L3 (format spec) +0.017, L3→L4 (definitions) +0.030, L4→L5 (persona) −0.020 (slightly harmful). A single well-chosen instruction captures most of the quality benefit. Output length correspondingly drops from 85.1 tokens (L1, verbose explanation) to 2.1 tokens (L3+, just the label) as format specification takes effect—the prompt teaches the model the output schema, and once learned, further elaboration adds nothing.

Product extraction (distributed). Gains accumulate across multiple layers: L1→L2 (field names) +0.054 (32% of gain), L6→L7 (worked example) +0.067 (40% of gain). The two schema-defining layers together account for 72% of total improvement. The worked example is critical because product extraction requires demonstrating a complex output format (4-field JSON) that cannot be fully communicated through format names alone.

Math reasoning (non-monotonic). L3 (“Give only the final numerical answer”) causes a dramatic quality collapse: L2= 0.967 → L3= 0.601 ($\Delta = -0.366$), across all seven models and both judges. L4 restores step-by-step reasoning and quality recovers immediately (L4= 0.964). This non-monotonicity demonstrates that prompt content can be actively harmful—certain instructions suppress chain-of-thought reasoning, which is critical for math accuracy (Wei et al., 2022). Excluding L3, the remaining trajectory is flat (L1= 0.951, L7= 0.970; $\Delta = +0.019$), consistent with ceiling-at-L1.

QA (insensitive). Total marginal gain is essentially zero (L1→L7 $\Delta = -0.035$). Every layer beyond bare input either adds nothing or slightly hurts—models already score 0.89–0.95 at L1.

Prompt-sensitivity gradient. These profiles define a four-level gradient: *concentrated* (classification—one layer captures most gain), *distributed* (product extraction—gains across multiple layers), *diffuse* (instruction following—small gains across many layers, largest L5→L6 = +0.010), and *insensitive* (QA, math, summarisation—near-zero or negative total gain). Curve fitting detects the distributed pattern (product extraction: 3/7 original significant, 13/15 in ablation) but not the concentrated pattern (classification: 4/7 original, 2/25 in ablation). Marginal contribution analysis complements curve fitting by detecting both.

4.5. Ceiling Stratification Explains Non-Saturation

Why do QA and math show no saturation? Stratifying by Level-1 quality (≥ 0.85 threshold) reveals a ceiling effect: across models, 16–20 of 20 QA examples already score above 0.85 at L1, leaving too few below-ceiling examples for reliable curve fitting. The above-ceiling subset shows flat-to-declining quality (e.g., gemini-flash: L1= 0.94, L7= 0.91; mean $\Delta = -0.035$), consistent with extra layers adding noise to already-correct responses.

Math shows the same ceiling pattern (17–19 of 20 examples above 0.85 at L1) but with the L3 non-monotonicity superimposed.

This confirms that QA and math “non-saturation” is ceiling-at-L1: models already have sufficient parametric knowledge from the bare question. Classification and product extraction start below ceiling because the prompt must communicate format requirements that the model cannot infer from the input alone.

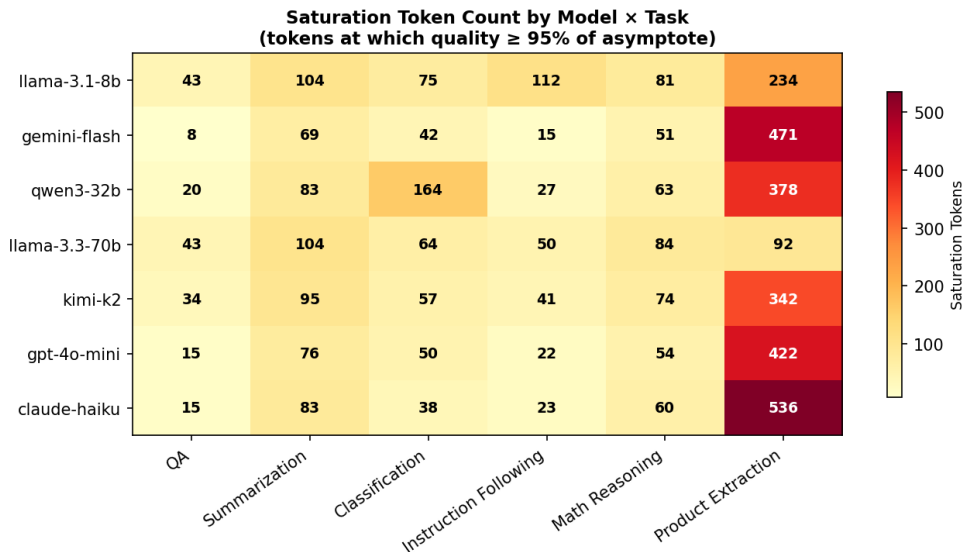


Figure 3: Saturation token counts (heatmap, model $\times$ task). Darker shading indicates more tokens needed for saturation. Cells for non-significant fits are shown in grey.

4.6. Model Capability Modulates Saturation

Within classification, API-tier models (gemini-flash, gpt-4o-mini) saturate at lower token counts (≈ 42 – 50 tokens) than the 70B Llama model (64 tokens): models that can infer the task from less context reach the quality plateau sooner. Within product extraction, the pattern reverses: API-tier models saturate *later* (claude-haiku at 536 tokens vs. llama-3.3-70b at 92 tokens) but reach *higher* asymptotic quality. For tasks where the model must *infer* what to do (classification), more capable models need less prompt. For tasks where the prompt provides *actionable format specifications* (extraction), more capable models leverage more of them, continuing to improve where smaller models plateau early at lower quality.

Sub-8B extension. To test whether capability-modulated saturation extends below the 8B floor, we evaluate Llama-3.2-1B on classification and product extraction (332 runs). The 1B model starts significantly lower at L1 (classification: $q = 0.467$ vs. 0.780 for 8B) and gains $1.58\times$ more from prompt elaboration (L1 $\rightarrow$ L7 $\Delta = +0.300$ vs. $+0.190$). For product extraction, the 1B model *completely fails* at L1–L2 ($q = 0.000$), then jumps to $q = 0.776$ at L3 when format specifications are introduced. These results extend the capability-modulated trend monotonically from 1B to 70B+ API-tier models: smaller models are more prompt-sensitive—they require more elaboration and benefit more from it—but saturate at lower asymptotic quality.

4.7. Dual-Judge Agreement

Table 7 reports inter-judge agreement. Agreement is strong where it matters most: classification ($r = 0.835$, MAE= 0.044) and math reasoning ($r = 0.859$, MAE= 0.033). For QA and product extraction, Pearson r is low, but this reflects ceiling compression rather than pattern disagreement: the gemini judge assigns scores > 0.9 to 98.8% of QA responses

Table 7: Inter-judge agreement between gpt-4o-mini (primary) and gemini-2.0-flash (second judge).

Task	n	Pearson r	MAE	Primary mean	Gemini mean
Classification	980	0.835	0.044	0.922	0.961
Math reasoning	978	0.859	0.033	0.910	0.927
QA	980	0.287	0.086	0.912	0.995
Product extraction	977	0.247	0.150	0.835	0.976

and 88.7% of product extraction responses, leaving almost no variance to correlate. Both judges agree on the L1→L7 direction for 5/7 models on classification and 4/7 on product extraction; the 3 product extraction non-agreements involve gemini deltas of ≤ 0.003 .

4.8. Threshold Sensitivity

For classification, saturation points are stable across threshold choices: the median 90%-to-95% shift across all 7 models is 3 tokens, and 6 of 7 models shift by fewer than 11 tokens. One outlier—qwen3-32b—follows a logarithmic rather than sigmoid trajectory, producing a 95-token shift. This model uniquely benefits from the guidelines layer (L6), suggesting it integrates detailed instructions more gradually. Its threshold-free knee estimate (42 tokens) falls close to the other models’, confirming similar saturation onset despite the different curve shape.

4.9. Replication on Larger Dataset

The 200-example replication confirms the main-study patterns. Classification again exhibits a clear sigmoid relationship, with the estimated saturation point at approximately 64 tokens—consistent with the main experiment. QA shows no significant relationship between prompt length and quality, reinforcing the insensitive classification.

5. Discussion

5.1. Information Sufficiency, Not Token Budgets

The convergence of three independent experiments—the core saturation study, the randomised ablation, and the padding control—supports a single mechanism: quality saturates once *task-critical information* has been communicated. The saturation *shape* (quality rising then plateauing) is robust across prompt orderings, but the specific token count at which it occurs is not—it shifts with which layers appear early. Irrelevant tokens at matched counts produce flat or declining quality. The practical implication is that prompt optimisation should focus on *which mechanisms to include*, not on *how many tokens to budget*.

5.2. Schema Compliance as a Mechanism

Three converging lines of evidence support the hypothesis that structured tasks saturate because quality is governed by schema compliance—the model’s ability to produce a response in the required format:

1. **Marginal contributions:** The schema-defining layers dominate quality gain. For classification, L1→L2 (task label) and L2→L3 (format spec) account for 61% of total gain. For product extraction, L6→L7 (worked example demonstrating output schema) accounts for 40%.
2. **Ceiling stratification:** Open-ended tasks (QA, math) reach ceiling at L1 precisely because they do *not* require schema compliance—the model already knows what to output.
3. **Output compression:** Classification output drops from 85 tokens (L1, verbose explanation) to 2 tokens (L3+, just the label) as format specification takes effect. Quality tracks this compression.

We present schema compliance as a hypothesis supported by converging evidence rather than a directly tested mechanism. Mechanistic interpretability analyses of how models process elaboration levels could provide a direct test.

5.3. Harmful Prompt Content

The math L3 finding—instructing models to “give only the final numerical answer” destroys quality by -0.366 across all seven models and both judges—demonstrates that prompt content can be actively harmful. This extends beyond the “diminishing returns” framing: certain instructions are not merely unhelpful but counterproductive. The standard sigmoid saturation model does not capture tasks where intermediate prompt layers are detrimental, and this non-monotonicity is important for practitioners to be aware of.

5.4. Practical Guidelines

The prompt-sensitivity gradient provides concrete, task-specific guidance:

- **Classification (concentrated):** Specify the task label and output format (≈ 50 tokens). Adding worked examples ($\approx 300+$ tokens) provides no benefit for most models.
- **Structured extraction (distributed):** Include a worked example demonstrating the expected output format. Models vary substantially (92–536 tokens); smaller models require more schema detail while larger or API-tier models infer the schema from minimal cues.
- **Instruction following (diffuse):** Small gains spread across many layers; no single mechanism dominates.
- **QA and math reasoning (insensitive):** The bare question suffices. Invest in retrieval-augmented generation (for QA) or chain-of-thought formatting (for reasoning) rather than instruction elaboration. *Avoid* suppressing step-by-step reasoning for math.

5.5. Relationship to Prompt Compression

Our work and prompt compression methods (LLMLingua, LLMLingua-2) operate at different stages of the prompt lifecycle. Compression decides what to keep *given a written prompt*; our marginal contribution analysis decides what to *write in the first place*. For classification, LLMLingua-2 might compress L7 (≈ 180 tokens) to ≈ 36 tokens by removing filler—but our analysis shows L1 $\rightarrow$ L2 (adding ≈ 5 tokens of task label) already captures 49% of the quality gain, suggesting the L7 elaboration was largely unnecessary content rather than redundantly worded content. The two approaches are complementary and stack: author the right mechanisms (our work), then compress the wording within them (prompt compression).

5.6. Limitations

Statistical power. With $n = 7$ data points per curve, the F-test has limited power. Borderline cases (classification for llama-3.1-8b at $p = 0.079$; product extraction for kimi-k2 at $p = 0.093$) are plausibly true positives that the marginal contribution analysis confirms as prompt-sensitive. Future work should use finer-grained level spacing (15–20 levels).

Classification domain breadth. Classification results draw from sentiment analysis only. Product extraction provides a second structurally distinct task with the same qualitative saturation pattern, but extending to additional classification domains (topic, intent, category) would strengthen generalisability.

Single-turn scope. The study focuses on single-turn instruction tasks. Agentic, RAG, and multi-turn settings may exhibit different saturation dynamics—our mechanism-level findings yield testable predictions for these settings (e.g., tool-call format specifications may exhibit concentrated sensitivity analogous to classification).

English only. All prompts and examples are in English. Saturation behaviour may differ for low-resource languages where prompt elaboration is more necessary to disambiguate tasks.

Model deprecation. kimi-k2 was deprecated by Groq between the main study (March 2026) and the ablation experiments (May 2026), producing 404 errors on all 490 ablation attempts. Main-study data from kimi-k2 is retained (recorded outputs are immutable), and the ablation analyses are restricted to the 5 models with complete data.

Post-hoc gradient. The prompt-sensitivity gradient (concentrated $\rightarrow$ distributed $\rightarrow$ diffuse $\rightarrow$ insensitive) is a post-hoc descriptive framework supported by three independent analyses (ceiling stratification, marginal contributions, ablation), not a pre-registered taxonomy. Validation as a predictive framework requires testing on new tasks.

6. Conclusion

We have presented a controlled study of prompt sensitivity across six tasks, eight models (1B–frontier), and 17,378 LLM evaluations. Through a combination of additive prompt levels, randomised layer-ordering ablation, and irrelevant-padding controls, we demonstrate

that response quality is governed by **information sufficiency**: quality saturates once task-critical content has been communicated, irrespective of raw token count or layer ordering.

Tasks differ in a **prompt-sensitivity gradient**: classification gain is concentrated in a single mechanism (task label), product extraction gain is distributed across multiple mechanisms, and QA/math are insensitive (already at ceiling from the bare input). This gradient, supported by marginal contribution analysis, ceiling stratification, and dual-judge validation, replaces the binary structured–open framing and provides concrete guidance for prompt authoring: identify which prompt mechanisms matter for your task type, include those, and stop.

Future directions. Extending the study to additional structured tasks (code generation with type annotations, data-to-text generation) would test whether the gradient generalises. Mechanistic interpretability analyses could directly test the schema-compliance hypothesis. Studying saturation in multi-turn and agentic settings—where context accumulates across turns and tool-call format specifications may exhibit concentrated sensitivity—would address the increasingly common deployment pattern of long-horizon LLM agents.

References

- Tom Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 2020.
- Yiming Chen et al. Evaluating scoring bias in LLM-as-a-judge. *arXiv preprint*, 2025.
- Alexis Chevalier et al. Adapting language models to compress contexts. *Proceedings of EMNLP*, 2023.
- Joong Ho Choi et al. CompactPrompt: A unified pipeline for prompt and data compression in LLM workflows. *arXiv preprint arXiv:2510.18043*, 2025.
- Jordan Hoffmann et al. Training compute-optimal large language models. *Advances in Neural Information Processing Systems*, 35, 2022.
- Cheng-Ping Hsieh et al. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
- Huiqiang Jiang et al. LLMingua: Compressing prompts for accelerated inference of large language models. *Proceedings of EMNLP*, 2023.
- Jared Kaplan et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Mosh Levy et al. Same task, more tokens: The impact of input length on the reasoning performance of large language models. *arXiv preprint arXiv:2402.14848*, 2024.
- Wei Li et al. Same input, different scores: Systematic calibration differences across LLM judges. *arXiv preprint*, 2026.
- Wenwu Li, Xiangfeng Wang, Wenhao Li, and Bo Jin. A survey of automatic prompt engineering: An optimization perspective. *arXiv preprint arXiv:2502.11560*, 2025.

- Nelson F. Liu et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.
- Moran Mizrahi et al. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12, 2024.
- Zhuoshi Pan et al. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*, 2024.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of EMNLP*, pages 2383–2392, 2016.
- Pranab Sahoo et al. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.
- Sander Schulhoff et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, pages 1631–1642, 2013.
- Albert Webson and Ellie Pavlick. Do prompt-based models really understand the meaning of their prompts? *Proceedings of NAACL*, 2022.
- Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.
- Lianmin Zheng et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Yongchao Zhou et al. Large language models are human-level prompt engineers. *International Conference on Learning Representations*, 2023.

Appendix A. Full Prompt Level Specifications

A.1. Classification

1. Bare text input only.
2. Add task label: “Classify the sentiment of the following text as Positive, Negative, or Neutral.”
3. Add format constraint: “Respond with only the single label word.”
4. Add label definitions: Positive = expresses approval or satisfaction; Negative = expresses dissatisfaction or criticism; Neutral = neither or mixed.
5. Add persona: “You are an expert sentiment analyst.”

6. Add guidelines: rules for ironic text, mixed sentiment, and domain jargon.
7. Add worked example: a complete input/output example with explanation.

A.2. Product Extraction

1. Bare product listing text only.
2. List required output fields: name, price, brand, category.
3. Specify JSON output format with exact key names.
4. Define each field: name = product title, price = numeric value without currency symbol, brand = manufacturer name, category = product type.
5. Add persona (expert product data analyst) and missing-field instruction (use null for absent fields).
6. Add extraction guidelines for ambiguous cases: implied brand names, price ranges, composite product names.
7. Add full worked example: verbose product description with correct JSON output.

A.3. Question Answering

1. Question only.
2. Add task label: “Answer the following question.”
3. Add format guidance: “Be concise and direct.”
4. Add domain context appropriate to the question.
5. Add expert persona.
6. Add uncertainty instruction: express uncertainty explicitly if unsure.
7. Add worked example.

A.4. Math Reasoning

1. Problem statement only.
2. Add task label: “Solve the following math problem.”
3. Add answer format: “Give a single numeric answer.”
4. Add show-work instruction: “Show your reasoning step by step.”
5. Add math tutor persona.
6. Add step-by-step guidelines and common error warnings.
7. Add worked example with full solution.

A.5. Summarisation

1. Article text only.
2. Add task label: “Summarise the following article.”
3. Add length target: “Write a summary of approximately 50–80 words.”
4. Add content guidance: include who, what, when, where.
5. Add journalist persona.
6. Add quality guidelines: avoid redundancy, preserve key facts.
7. Add worked example: article with reference summary.

A.6. Instruction Following

1. Topic prompt only (“List benefits of exercise”).
2. Add task label and constraints explicitly stated.
3. Add format specification mirroring the constraint type.
4. Add constraint definitions (e.g., bullet = “-” or “*” prefix).
5. Add writing expert persona.
6. Add formatting guidelines and error examples.
7. Add worked example satisfying all constraints.

Appendix B. LLM Judge Rubric

All responses are scored by the primary judge (`gpt-4o-mini`) and the second judge (`gemini-2.0-flash`) using the following prompt template. The `{rubric}` placeholder is filled with a task-specific rubric (Table 8).

You are an expert judge evaluating an AI assistant’s response.

Task type: `{task_type}`

Evaluation guidance: `{rubric}`

Reference answer (if available): `{ground_truth}`

AI response to evaluate:

`{response}`

Rate the response on these 4 dimensions, each from 1 (very poor) to 5 (excellent):

- correctness: Does the response accurately answer the task?
- completeness: Is the response complete and thorough?
- reasoning: Is the reasoning or logic sound and clear?
- conciseness: Is the response appropriately concise?

Respond ONLY with valid JSON in this exact format:

```
{"correctness": <1-5>, "completeness": <1-5>, "reasoning": <1-5>, "conciseness": <1-5>}
```

Table 8: Task-specific rubrics provided to both LLM judges.

Task	Rubric
Classification	Focus on Correctness (does the classification match the reference label?). Conciseness rewards direct labelling without padding. Reasoning Quality assesses any justification given. Completeness checks if the classification is unambiguous.
Product Extraction	Focus on Correctness (do the extracted fields match the reference values?). Completeness checks if all 4 fields (name, price, brand, category) are present. Reasoning Quality is less relevant; focus on extraction accuracy. Conciseness rewards clean JSON output without extra text.
QA	Focus heavily on Correctness (does the answer match the reference?). Completeness checks if the answer is fully given. Reasoning Quality assesses logical justification. Conciseness penalises unnecessary verbosity.
Math Reasoning	Focus on Correctness (does the final numerical answer match the reference?). Completeness checks if work is shown. Reasoning Quality assesses whether the solution steps are logical. Conciseness penalises unnecessary verbosity beyond the solution.
Summarisation	Focus on Completeness (coverage of key points from the reference). Correctness checks factual accuracy. Conciseness rewards brevity over length. Reasoning Quality assesses coherence and structure.
Instr. Following	Focus on Completeness (are all stated constraints satisfied?). Correctness checks format and structural adherence. Conciseness rewards appropriate brevity. Reasoning Quality assesses overall response clarity.

Appendix C. Per-Model Saturation Statistics

Table 9 reports fit type, R^2 , F-test p -value, and saturation token count for all 42 (model, task) pairs in the main study.

Appendix D. Full Prompt Walkthrough: Classification

Below is the complete prompt text at each of the seven additive levels for a single classification input: *“The product works great and I’m very happy with my purchase.”* Each level is a strict superset of the previous.

Level 1 (bare input).

Classify: The product works great and I’m very happy with my purchase.

Level 2 (+ task label).

Classify sentiment as positive, negative, or neutral: The product works great and I’m very happy with my purchase.

Table 9: Complete saturation statistics for all model-task pairs. Bold: F-test $p < 0.05$. Fit = best-fit curve type (log = logarithmic, sig = sigmoid).

Model	Task	Fit	R ²	p	T^*
llama-3.1-8b	QA	log	0.291	0.503	43
llama-3.1-8b	Summarisation	sig	0.161	0.896	104
llama-3.1-8b	Classification	sig	0.866	0.079	75
llama-3.1-8b	Instr. Following	sig	0.929	0.031	112
llama-3.1-8b	Math Reasoning	sig	0.125	0.928	81
llama-3.1-8b	Product Extraction	sig	0.720	0.229	234
gemini-flash	QA	sig	0.404	0.622	8
gemini-flash	Summarisation	sig	0.760	0.185	69
gemini-flash	Classification	sig	0.936	0.027	42
gemini-flash	Instr. Following	log	0.067	0.871	15
gemini-flash	Math Reasoning	sig	0.205	0.853	51
gemini-flash	Product Extraction	sig	0.830	0.113	471
qwen3-32b	QA	log	0.690	0.096	20
qwen3-32b	Summarisation	sig	0.106	0.944	83
qwen3-32b	Classification	log	0.753	0.061	164
qwen3-32b	Instr. Following	sig	0.465	0.544	27
qwen3-32b	Math Reasoning	sig	0.217	0.840	63
qwen3-32b	Product Extraction	log	0.930	0.005	378
llama-3.3-70b	QA	log	0.065	0.875	43
llama-3.3-70b	Summarisation	log	0.290	0.503	104
llama-3.3-70b	Classification	sig	0.957	0.015	64
llama-3.3-70b	Instr. Following	log	0.379	0.385	50
llama-3.3-70b	Math Reasoning	sig	0.282	0.769	84
llama-3.3-70b	Product Extraction	sig	0.943	0.023	92
kimi-k2	QA	log	0.097	0.816	34
kimi-k2	Summarisation	log	0.664	0.113	95
kimi-k2	Classification	sig	0.955	0.016	57
kimi-k2	Instr. Following	log	0.027	0.947	41
kimi-k2	Math Reasoning	sig	0.339	0.702	74
kimi-k2	Product Extraction	log	0.695	0.093	342
gpt-4o-mini	QA	log	0.646	0.125	15
gpt-4o-mini	Summarisation	log	0.356	0.414	76
gpt-4o-mini	Classification	sig	0.977	0.006	50
gpt-4o-mini	Instr. Following	log	0.386	0.377	22
gpt-4o-mini	Math Reasoning	sig	0.178	0.879	54
gpt-4o-mini	Product Extraction	sig	0.877	0.071	422
claude-haiku	QA	log	0.618	0.146	15
claude-haiku	Summarisation	log	0.824	0.031	83
claude-haiku	Classification	sig	0.646	0.317	38
claude-haiku	Instr. Following	log	0.071	0.863	23
claude-haiku	Math Reasoning	sig	0.228	0.829	60
claude-haiku	Product Extraction	sig	0.980	0.005	536

Level 3 (+ format spec).

Classify sentiment as positive, negative, or neutral. Respond with only the label: The product works great and I'm very happy with my purchase.

Level 4 (+ definitions).

Classify the sentiment of the following text as positive, negative, or neutral. Respond with only the label.

Definitions:

- positive: overall favorable or optimistic tone
- negative: overall unfavorable or critical tone
- neutral: balanced, factual, or no clear sentiment

Text: The product works great and I'm very happy with my purchase.

Level 5 (+ persona).

You are an expert sentiment analyst. Classify the sentiment of the following text as positive, negative, or neutral. Respond with only the label.

Definitions:

- positive: overall favorable or optimistic tone
- negative: overall unfavorable or critical tone
- neutral: balanced, factual, or no clear sentiment

Text: The product works great and I'm very happy with my purchase.

Level 6 (+ guidelines).

You are an expert sentiment analyst. Classify the sentiment of the following text as positive, negative, or neutral.

Rules:

1. Respond with ONLY one word: positive, negative, or neutral
2. Base your judgment on the overall tone, not individual words
3. Positive: clearly favorable, optimistic, praising, or satisfied
4. Negative: clearly unfavorable, critical, pessimistic, or dissatisfied
5. Neutral: factual reporting, balanced views, or no discernible sentiment
6. If mixed, choose the dominant sentiment; if equal, use neutral
7. Take text at face value; do not attempt sarcasm detection

Text: The product works great and I'm very happy with my purchase.

Level 7 (+ worked example).

You are an expert sentiment analyst. Classify the sentiment of the following text as positive, negative, or neutral.

Rules:

1. Respond with ONLY one word: positive, negative, or neutral
2. Base your judgment on the overall tone, not individual words
3. Positive: clearly favorable, optimistic, praising, or satisfied
4. Negative: clearly unfavorable, critical, pessimistic, or dissatisfied
5. Neutral: factual reporting, balanced views, or no discernible sentiment
6. If mixed, choose the dominant sentiment; if equal, use neutral
7. Take text at face value; do not attempt sarcasm detection

Example:

Text: The product works great and I'm very happy with my purchase.
 Label: positive
 Now classify:
 Text: The product works great and I'm very happy with my purchase.

Model responses (gpt-4o-mini).

- **L1** ($q = 0.90$): “This review can be classified as **Positive**.”—correct label, but verbose explanation; format does not match.
- **L3** ($q = 1.00$): “Positive”—just the label, format matches.
- **L5–L7** ($q = 1.00$): “positive”—no further improvement despite 3–6× more prompt tokens.

Appendix E. Full Prompt Walkthrough: Product Extraction

Input: *“Check out the Nike Air Max 270 - the ultimate sneaker for style and comfort. Priced at just \$150, these shoes feature a large Air unit in the heel for all-day cushioning.”*

Level 1 (bare input).

Extract information from this product listing:
 Check out the Nike Air Max 270 - the ultimate sneaker for style and comfort.
 Priced at just \$150, these shoes feature a large Air unit in the heel for
 all-day cushioning.

Level 2 (+ field names).

Extract the following information from this product listing: name, price,
 brand, category.
 Product: Check out the Nike Air Max 270 - the ultimate sneaker for style
 and comfort. Priced at just \$150, these shoes feature a large Air unit in
 the heel for all-day cushioning.

Level 3 (+ JSON format).

Extract the following information from this product listing and return it
 as JSON with these exact keys: name, price, brand, category.
 Product: Check out the Nike Air Max 270 [...]

Level 4 (+ field definitions).

[...as L3, plus:]
 Field definitions:
 - name: the full product title
 - price: numeric value without currency symbol
 - brand: the manufacturer or brand name
 - category: the product type or category
 Product: Check out the Nike Air Max 270 [...]

Level 5 (+ persona & null handling).

You are an expert product data analyst. [...]
 If a field is not present in the listing, use null.
 Product: Check out the Nike Air Max 270 [...]

Level 6 (+ edge-case guidelines).

[...as L5, plus:]
 Guidelines for ambiguous cases:
 - For implied brand names, infer from product name if possible
 - For price ranges, use the lower bound
 - For composite product names, include the full name as listed
 Product: Check out the Nike Air Max 270 [...]

Level 7 (+ worked example).

[...as L6, plus:]
 Example:
 Product: ‘‘Premium Sony WH-1000XM5 wireless headphones, now on sale for \$298. Features industry-leading noise cancellation.’’
 Output: {"name": "Sony WH-1000XM5", "price": "298", "brand": "Sony", "category": "headphones"}
 Now extract from:
 Product: Check out the Nike Air Max 270 [...]

Model responses (gemini-flash).

- **L1** ($q = 0.70$): Bulleted list with extra fields (Product Type, Key Features, Availability)—correct information but wrong format.
- **L3** ($q = 0.80$): JSON with correct keys, but ‘‘Air Max 270’’ instead of ‘‘Nike Air Max 270’’ for the name field.
- **L7** ($q = 1.00$): {"name": "Nike Air Max 270", "price": "150", "brand": "Nike", "category": "shoes"}—the worked example teaches the exact output schema, producing the quality jump that marginal analysis identifies as 40% of total gain.

Appendix F. Output Length Control

Output length varies across levels. We computed partial correlations controlling for output length to confirm the prompt→quality relationship is not an artefact of output compression.

The prompt→quality relationship persists after controlling for output length (positive partial r for all three tasks).

Appendix G. Replication Experiment

The 200-example replication on SST-2 (classification) and SQuAD (QA) confirms the main-study findings. Classification saturates at approximately 64 tokens with a narrow bootstrap CI, while QA shows no statistically significant relationship between prompt length and quality.

Table 10: Partial correlations controlling for output length.

Task	$r(\text{output, quality})$	partial $r(\text{prompt, quality} \mid \text{output})$
Classification	-0.186	0.106
Product extraction	-0.313	0.447
Instr. following	0.616	0.209